

FuncR: Binary Function Name Recovery via Graph-Based Multi-Context Embeddings and Retrieval

Ananta Dian Pradipta
New Jersey Institute of Technology
Newark, NJ, USA
adp232@njit.edu

Robert Blacha
New Jersey Institute of Technology
Newark, NJ, USA
rb236@njit.edu

Zhihao Lin
New Jersey Institute of Technology
Newark, NJ, USA
zl74@njit.edu

ABSTRACT

Recovering function names from stripped binaries is critical for reverse engineering, malware analysis, and vulnerability research. We present FuncR, a graph neural network-based system that learns function representations by fusing multiple context sources—external library calls, callee signatures, and caller signatures—through a cascaded gated fusion mechanism with conditional bypass. Our ablation study reveals a novel *ext-call paradox*: external call information alone improves performance on seen binaries but *degrades* generalization to unseen packages by 4.3 percentage points; inter-procedural callee/caller context is required for disambiguation. Furthermore, we demonstrate that nearest-neighbor retrieval over learned encoder embeddings outperforms autoregressive sequence decoding, improving exact match by 4.6 percentage points on unseen packages with zero retraining. This finding confirms that binary function naming is fundamentally a *recognition* task rather than a generation task—the encoder representation, not the decoder, is the key component. Evaluated on 87,724 functions from 40 packages, FuncR achieves 0.804 F1 on a held-out test set (8,973 functions) and 0.625 F1 across 11 completely unseen packages (12,688 functions), outperforming BLens (0.46 F1) and SYMGEN (0.38 F1) in cross-project settings.

KEYWORDS

binary analysis, function name recovery, graph neural networks, gated fusion, retrieval-augmented inference

1 INTRODUCTION

Software reverse engineering is a fundamental activity in cybersecurity, requiring analysts to understand the behavior of compiled binary programs without access to source code. Modern compilers strip all symbolic information—function names, variable names, type annotations—from release binaries, leaving analysts with thousands of unnamed functions (e.g., `sub_4a30`, `FUN_00105c10`) that must be manually identified. Recovering meaningful function names dramatically accelerates program comprehension: knowing that `sub_4a30` is `xmalloc` immediately conveys its purpose.

Prior work on automated function name recovery has explored statistical approaches [1], graph neural networks [2], language model pre-training [3], and large language model fine-tuning [5]. However, two fundamental challenges remain underexplored: (1) how to effectively combine multiple sources of contextual information—code structure, library calls, and inter-procedural relationships—without degrading

generalization, and (2) whether autoregressive name generation is the right output paradigm, given that function names are drawn from a relatively small vocabulary of real names.

We address these challenges with FuncR, a system that makes three contributions:

- (1) **Multi-context gated fusion with conditional bypass.** We propose a 3-stage cascaded fusion architecture that sequentially incorporates external library calls, callee signatures, and caller signatures. Each stage uses a learned sigmoid gate, and stages are *bypassed* when no context exists, preventing the mode collapse observed in 47% of predictions without this mechanism.
- (2) **The ext-call paradox.** Our ablation reveals that external call information alone improves test F1 by +0.077 but *hurts* generalization to unseen packages by −4.3 percentage points. Many unrelated functions share similar library call patterns; without inter-procedural callee/caller context for disambiguation, the model overfits to these patterns. Adding callee/caller context resolves the paradox (+12.8pp demo EM). This finding demonstrates that naive feature addition can degrade generalization, and explains *why* the cascaded fusion architecture is necessary—not just effective.
- (3) **Binary function naming is recognition, not generation.** We show that 0 out of 1,873 unseen function names are correctly predicted—every correct prediction matches a name seen in training. *k*-NN retrieval over encoder embeddings confirms this: it outperforms the autoregressive decoder by +2.0pp test EM and +4.6pp demo EM with zero retraining, because retrieval constrains outputs to real names and eliminates the 43.8% “phantom” predictions the decoder produces. This establishes the encoder representation as the key component, not the decoder.

Evaluated on 87,724 functions from 40 packages, FuncR achieves 0.804 F1 on a held-out test set (8,973 functions) and 0.625 F1 across 11 completely unseen packages (12,688 functions), competitive with BLens [4] (0.46) and SYMGEN [5] (0.38) in cross-project settings.

The practical importance of automated function name recovery extends across multiple domains of cybersecurity. In malware analysis, analysts must rapidly triage thousands of functions in novel samples to identify malicious behavior; meaningful function names reduce analysis time from hours to minutes. In vulnerability research, understanding stripped

firmware binaries is essential for discovering exploitable bugs in embedded devices, routers, and IoT systems where source code is unavailable. In Capture-the-Flag (CTF) competitions and forensic investigations, reverse engineers routinely encounter stripped binaries where even partial name recovery provides critical footholds for understanding program logic. Commercial tools such as IDA Pro and Ghidra provide manual annotation workflows, but cannot automatically recover names—our system addresses this gap.

The rest of this paper is organized as follows. Section II formally defines the problem and threat model. Section III surveys related work on function name recovery, binary code representation, and graph neural networks for binary analysis. Section IV presents the FuncR system design, including preprocessing, encoder architecture, multi-context fusion, self-supervised pre-training, and inference strategies. Section V evaluates FuncR with an ablation study, cross-project experiments, per-package breakdown, comparison with published systems, and detailed error analysis. Section VI discusses implications, and Section VII identifies limitations. Section VIII concludes.

2 PROBLEM DEFINITION AND THREAT MODEL

2.1 Problem Definition

We formalize binary function name recovery as follows. Given a stripped binary B containing n functions $\{f_1, f_2, \dots, f_n\}$, where each function f_i is represented by its control flow graph $G_i = (V_i, E_i)$ with basic blocks as nodes and control flow edges, the goal is to predict a human-readable name $\hat{n}_i$ for each function that matches the developer-assigned name n_i from the original source code.

Each function name is decomposed into a sequence of sub-tokens using a vocabulary $\mathcal{V}$ of 2,642 tokens: $n_i = (t_1, t_2, \dots, t_L)$ where $t_j \in \mathcal{V}$ and $L \leq 20$. For example, `quotearg_buffer_restyled` becomes the sub-token sequence `[quote, arg, _, buffer, _, re, styled]`. The prediction task is thus a sequence generation problem conditioned on the function’s binary representation.

In addition to the CFG, each function may have three forms of contextual information: (1) external library calls $\mathcal{E}_i$ —the set of PLT/IAT function names called by f_i (e.g., `malloc`, `fprintf`); (2) callee signatures $\mathcal{C}_i^{ee}$ —instruction-type token sequences of internal functions called by f_i ; and (3) caller signatures $\mathcal{C}_i^{er}$ —instruction-type token sequences of functions that call f_i . These contextual signals are optional: approximately 70% of functions have no external calls, and leaf functions have no callees.

2.2 Scope and Assumptions

We scope FuncR to the following setting:

- **Architecture:** x86-64 ELF binaries compiled for Linux.
- **Optimization levels:** O0 (no optimization) and O2 (standard optimization). We do not evaluate O1 or

O3, as O1 is rarely used in practice and O3 introduces aggressive inlining that duplicates function bodies.

- **Compilation:** GCC compiler. We do not evaluate Clang or MSVC, though the instruction-type tokenization is compiler-agnostic at the IR level.
- **Training corpus:** We assume access to a corpus of labeled binaries where ground-truth function names are known (obtained from debug builds). This is a standard assumption in the literature [1–5].
- **No obfuscation:** We assume binaries are not deliberately obfuscated. Obfuscation techniques (control flow flattening, opaque predicates, symbol stripping with anti-disassembly) would require additional pre-processing.

2.3 Evaluation Scenario

We evaluate in a *cross-project* setting: 11 complete packages (12,688 functions) are held out entirely from training and used only for evaluation. This is stricter than the cross-binary evaluation used by some prior work [2, 3], where different binaries of the *same* package may appear in both training and test. Cross-project evaluation tests whether the model generalizes to entirely new codebases with potentially novel naming conventions, and is the evaluation methodology advocated by SYMGEN [5].

We additionally maintain a held-out test set of 8,973 functions from 18 binaries of packages that *are* in training (but the specific binaries are not). This allows us to measure in-distribution performance separately from cross-project generalization.

3 RELATED WORK

3.1 Binary Function Name Recovery

He et al. [1] introduced DeBin, which uses BAP intermediate representation with probabilistic graphical models (Extra-Trees + CRF) to predict debug information including function names. While pioneering, DeBin’s statistical approach cannot capture the complex patterns that deep learning exploits. We use DeBin as the non-neural baseline in our ablation (Table II).

David et al. [2] proposed NERO, using pointer-aware slicing and GNNs on call graphs to predict function names, achieving F1=0.455. NERO’s key insight is that call-site context matters for naming. However, NERO only works for functions *with* external calls—a fundamental limitation since 70% of functions in typical binaries have none. Our conditional bypass mechanism (Section IV-D) addresses this directly: functions without calls use only code features, while functions with calls benefit from multi-context fusion.

Jin et al. [3] presented SymLM, which pre-trains on micro-execution traces and fuses caller/callee context for function name prediction (F1=0.585). SymLM’s “fusing encoder” concatenates context embeddings, whereas our gated fusion learns a per-function balance between code and context features with conditional bypass. More critically, SymLM requires dynamic analysis infrastructure (executing binaries in

an emulator), whereas FuncR operates purely on static analysis. Our ablation reveals the ext-call paradox (Section V)—a phenomenon SymLM does not analyze—showing that context fusion must be carefully designed to avoid degrading generalization.

Al-Kaswan et al. [4] introduced BLens, combining ensemble embeddings with contrastive captioning. While achieving 0.79 F1 per-binary, cross-project F1 drops to 0.46—a 42% degradation that highlights the generalization challenge we address. Xu et al. [5] fine-tuned CodeLlama with LoRA for function naming, importantly demonstrating that prior results were inflated 1.2–2.9 $\times$ by duplicate functions (cross-project F1=0.38 after proper deduplication). We follow SYMGEN’s deduplication methodology in our evaluation.

Jiang et al. [6] applied domain-adapted LLMs to function name inference, representing the trend toward large-scale language model approaches. GenNm [7] fine-tuned CodeGemma with SymPO preference optimization for variable name recovery, a related task demonstrating that preference-aligned generation improves binary symbol prediction. Our approach differs fundamentally: rather than scaling model size (billions of parameters), we invest in the *encoder representation* (25M parameters) and show that simple k -NN retrieval over these embeddings outperforms autoregressive generation—consistent with Khandelwal et al.’s finding that retrieval can outperform generation when representations are sufficiently good [1, 1].

3.2 Binary Code Representation Learning

Learning effective representations of binary code underpins all downstream binary analysis tasks. Xu et al. [8] pioneered GNN-based binary analysis with Gemini, using graph embedding networks for cross-platform binary similarity detection. We adopt their insight that CFG structure is informative, extending it with GAT attention and multi-context fusion. Ding et al. [9] proposed Asm2Vec, learning function embeddings via PV-DM random walks over assembly instructions. Unlike Asm2Vec’s raw instruction tokens, our V3 tokenization abstracts instructions into 1,510 semantic types, reducing vocabulary by 95% while preserving discriminative information.

Wang et al. [1] introduced jTrans, a jump-aware Transformer that incorporates control flow structure into positional encoding for binary similarity. Pei et al. [1] developed Trex, learning execution semantics from micro-traces. Both demonstrate the value of pre-training on binary code. We adopt this principle in our MLM + contrastive pre-training stage (Section IV-E), but operate on static BAP-IR rather than execution traces.

Zhu et al. [1] proposed CALLEE for recovering call graphs via contrastive learning with transfer learning, showing +28% F1 from pre-training on related tasks. Two of their findings directly support our design: (1) transfer learning from pre-training provides substantial gains, validating our encoder pre-training approach; (2) “loose symbolization” (semantic

type abstraction) outperforms strict tokenization, supporting our instruction-type tokenization.

3.3 GNNs for Binary Analysis

Graph neural networks are a natural fit for binary analysis, where control flow graphs (CFGs) provide structural information. Zhu et al. [1] applied GNNs to type inference on stripped binaries (TYGR), demonstrating that graph structure captures semantic properties beyond sequential instruction patterns. He et al. [1] proposed semantics-oriented graph representations for binary code similarity, arguing that “code is not natural language” and that graph-based approaches better capture program semantics than sequential models.

Our work extends GNN-based binary analysis in two ways: (1) we use graph attention networks (GAT) with learned attention over CFG edges, allowing the model to weight structurally important blocks; (2) we introduce a multi-context fusion mechanism that incorporates inter-procedural information (callee/caller signatures and library calls) not present in the CFG alone.

3.4 Retrieval-Augmented Generation

Our finding that k -NN retrieval outperforms autoregressive decoding connects to a growing body of work on retrieval-augmented methods in NLP. Khandelwal et al. [1] demonstrated that augmenting a neural language model with a nearest-neighbor search over cached hidden states (k NN-LM) significantly improves perplexity without any additional training. They extended this to machine translation (k NN-MT) [1], showing that retrieval-augmented translation outperforms the base model, particularly on domain-specific data. Borgeaud et al. [1] proposed RETRO, which retrieves relevant text chunks from a large database during generation, achieving performance comparable to models 25 $\times$ larger.

These works share a common insight with our findings: when the output space is drawn from a fixed set of real examples (function names in training, sentences in a corpus), retrieval over learned representations can outperform unconstrained generation. However, our setting differs in an important way: in k NN-LM/ k NN-MT, retrieval *augments* generation at the token level, whereas in our system, retrieval *replaces* generation entirely—we retrieve a complete function name rather than interpolating token-level distributions. This is because function names are atomic labels (each name is a single concept), not compositional sequences where token-level retrieval would help.

4 SYSTEM DESIGN

FuncR follows a 3-stage encoder-decoder pipeline (Figure 2): (1) a block encoder that produces per-basic-block embeddings from instruction-type tokens, (2) a graph encoder that aggregates block embeddings over the control flow graph, and (3) a cascaded gated fusion module that incorporates external calls, callee context, and caller context. At inference time, we use either a GRU decoder or k -NN retrieval to produce function names.

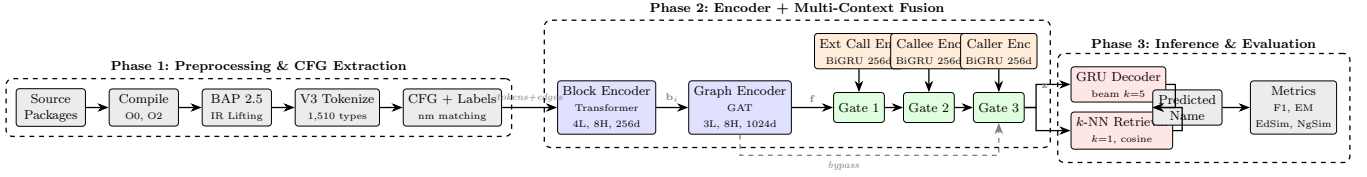


Figure 1: End-to-end FuncR pipeline. Phase 1 compiles source packages, lifts to BAP-IR, tokenizes instructions into 1,510 semantic types, and extracts CFGs with ground-truth labels. Phase 2 encodes basic blocks (Transformer), aggregates over the CFG (GAT), and fuses library call context, callee signatures, and caller signatures through cascaded gated fusion with conditional bypass. Phase 3 produces function names via either GRU beam search or k -NN retrieval, evaluated with sub-token F1, exact match, edit similarity, and n-gram similarity.

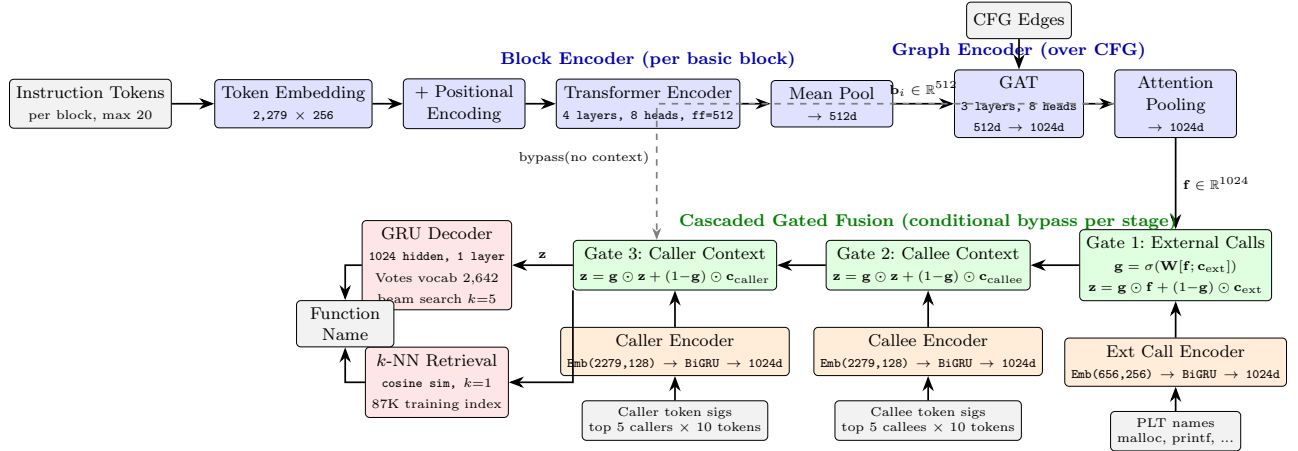


Figure 2: Detailed FuncR model architecture. The block encoder processes instruction-type tokens per basic block via a Transformer. The graph encoder aggregates block embeddings over the CFG using GAT with attention pooling. The cascaded gated fusion sequentially incorporates external library calls, callee signatures, and caller signatures—each stage is bypassed when the corresponding context is absent. The fused embedding z feeds either a GRU decoder (beam search) or k -NN retrieval.

4.1 Preprocessing

We compile source packages at optimization levels O0 and O2, producing both debug and stripped binaries. We use BAP 2.5.0 [1] to lift stripped binaries to BAP Intermediate Representation (BAP-IR), which provides a normalized, architecture-independent view of the binary code.

4.1.1 Instruction-Type Tokenization. Raw BAP-IR contains over 32,000 unique tokens, many of which are register names, immediate values, or memory addresses that vary across binaries. We classify each BAP-IR instruction into one of approximately 1,510 semantic types:

- **Calls:** CALL_malloc, CALL_printf, CALL_INTERNAL
- **Memory:** MEM_READ_32, MEM_WRITE_ARG_64, MEM_READ_GLOBAL_32
- **Flags:** FLAG_CF, COND_BRANCH_ZF
- **Constants:** ASSIGN_ZERO, ASSIGN_POW2, ASSIGN_ADDR (immediate value bucketing)
- **Control:** BRANCH, RETURN, COMPARE

This tokenization reduces vocabulary by 95% while preserving semantic distinctions. Our ablation shows a +1,750% F1 gain over raw BAP-IR tokens. The tokenization is deterministic and compiler-agnostic: it depends only on BAP-IR semantics, not on specific register assignments or memory layouts.

4.1.2 Dataset Construction. Our dataset is constructed through a multi-step pipeline. First, we compile each source package twice: a *debug* build (with `-g` flag) and a *stripped* build (with `strip --strip-all`). The debug build provides ground-truth function names via the `nm` utility, which extracts symbol table entries mapping addresses to names. The stripped build is the input to our model—it contains no symbolic information.

We lift each stripped binary using BAP 2.5.0 [1], which produces BAP-IR files containing per-function control flow graphs. BAP names functions using their entry addresses (e.g., `sub_4a30`). We match BAP function addresses to debug symbols via exact address matching, discarding functions that

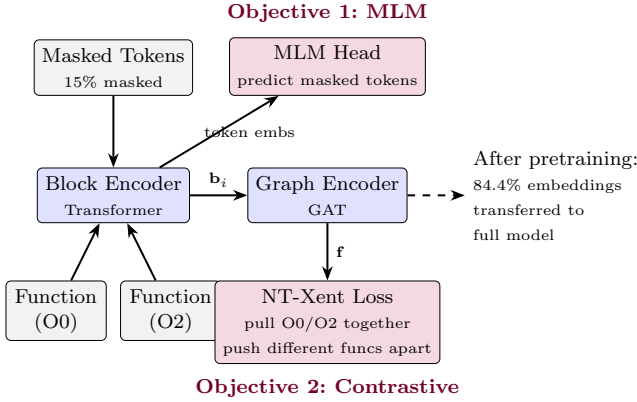


Figure 3: Self-supervised pretraining. Two objectives: (1) MLM predicts masked instruction tokens from block encoder token embeddings (before pooling), training the block encoder; (2) contrastive learning (NT-Xent) pulls together function-level embeddings (after graph encoder) of the same function at O0 and O2, training both encoders. After pretraining, 84.4% of token embeddings are transferred to the full model by matching token names.

cannot be matched (typically compiler-inserted functions like `_start` or `_libc_csu_init`).

Data is split at the *binary* level, not the function level, to prevent data leakage. A fixed split assignment file ensures reproducibility: 137 binaries for training, 17 for validation, and 18 for testing. Eleven entire packages (all their binaries) are held out as the demo set and never appear during training. This split design means that the same function (e.g., `xmalloc` from `glibc`) may appear in both training and test binaries—this is intentional, as it reflects the real-world scenario where shared library functions recur across packages.

4.1.3 Votes Name Tokenizer. Function names must be decomposed into sub-tokens for sequence prediction. Standard BPE tokenization produces poor results because function names follow naming conventions (snake_case, camelCase) that differ from natural language. We developed Votes, a 3-model voting tokenizer that achieves 95% lower out-of-vocabulary rate compared to BPE.

Votes works as follows: three independent segmentation models—a character bigram model, a frequency-based model, and a rule-based model (splitting on underscores, camelCase boundaries, and digit transitions)—each propose sub-token boundaries for a given function name. A boundary is accepted if at least 2 of 3 models agree. This produces a vocabulary of 2,642 sub-tokens that covers 99.7% of all function name tokens in the dataset, compared to 94.8% for BPE with a comparable vocabulary size. The reduced OOV rate directly improves exact match: a name cannot be correctly predicted if any of its sub-tokens are out of vocabulary.

For example, `quotearg_buffer_restyled` is tokenized as `[quote, arg, -, buffer, -, re, styled]` by Votes, whereas BPE

might produce `[quo, tear, g_buf, fer_rest, yled]`—losing semantic boundaries.

4.1.4 ENDBR64 Wrapper Resolution. In O0-compiled binaries, BAP lifts `endbr64; jmp target` sequences as separate 2-block functions. These are Intel CET (Control-flow Enforcement Technology) stubs, not real functions. We found that 96.4% of O0 functions were such wrappers, each containing only a single `CALL_INTERNAL` token. Our preprocessing detects these wrappers and replaces their graphs with the target function’s graph, raising O0 exact match from 1.6% to 41.5%.

4.2 Block Encoder

Each basic block b contains a sequence of up to 20 instruction-type tokens $(x_1, x_2, \dots, x_T)$ where $T \leq 20$. Blocks longer than 20 tokens are truncated; in practice, 95% of blocks have fewer than 20 tokens. We embed each token via a learned embedding table $\mathbf{E} \in \mathbb{R}^{2279 \times 256}$ and add sinusoidal positional encodings to preserve token order within the block:

$$\mathbf{h}_t^{(0)} = \mathbf{E}[x_t] + \text{PE}(t) \quad (1)$$

A Transformer encoder with 4 layers and 8 attention heads ($d_{\text{model}}=256$, $d_{\text{ff}}=512$) processes the token sequence with multi-head self-attention:

$$\mathbf{H}^{(\ell)} = \text{TransformerLayer}^{(\ell)}(\mathbf{H}^{(\ell-1)}) \quad (2)$$

We apply mean pooling over the final-layer token representations to produce the block embedding:

$$\mathbf{b}_i = \text{Linear} \left(\frac{1}{T} \sum_{t=1}^T \mathbf{h}_t^{(L)} \right) \in \mathbb{R}^{512} \quad (3)$$

We chose mean pooling over alternative aggregation strategies after extensive experimentation. Token-level attention pooling (a weighted sum using learned attention scores) achieved only 0.34 F1—significantly worse than mean pooling’s 0.58 F1. We hypothesize that attention pooling overfits to specific token positions, while mean pooling provides a more robust representation by treating all instruction types as equally important at the block level. This is consistent with the finding that instruction-type tokens are already semantically abstracted; further attention-based selection within a block provides diminishing returns.

A dropout rate of 0.15 is applied to the Transformer layers during training for regularization. Each function may contain up to 30 basic blocks; functions with fewer blocks are padded, and a block-level attention mask ensures padding blocks do not influence the graph encoder.

4.3 Graph Encoder

Given a function’s control flow graph $G = (V, E)$ where nodes are basic blocks with embeddings $\{\mathbf{b}_1, \dots, \mathbf{b}_{|V|}\}$, we apply a 3-layer Graph Attention Network (GAT) [2] with 8 attention heads per layer. Each GAT layer computes attention-weighted message passing along CFG edges:

$$\alpha_{ij}^{(k)} = \frac{\exp(\text{LReLU}(\mathbf{a}^{(k)\top} [\mathbf{W}^{(k)} \mathbf{b}_i \| \mathbf{W}^{(k)} \mathbf{b}_j]))}{\sum_{j' \in \mathcal{N}(i)} \exp(\text{LReLU}(\mathbf{a}^{(k)\top} [\mathbf{W}^{(k)} \mathbf{b}_i \| \mathbf{W}^{(k)} \mathbf{b}_{j'}]))} \quad (4)$$

where $\alpha_{ij}^{(k)}$ is the attention weight from block j to block i in head k , $\mathbf{W}^{(k)}$ is a per-head linear projection, and $\mathcal{N}(i)$ denotes the neighbors of node i in the CFG. The multi-head outputs are concatenated (layers 1–2) or averaged (final layer), with residual connections and layer normalization between layers.

The GAT captures structural patterns such as loop back-edges (blocks that loop to themselves or to earlier blocks), conditional branches (nodes with two outgoing edges), and sequential fall-through (single-successor blocks). These structural patterns are informative for naming; for instance, functions with deep loop nesting often implement search or sorting algorithms.

An attention-based readout pools over all block embeddings to produce the function-level embedding:

$$\mathbf{f} = \sum_{i=1}^{|V|} \beta_i \cdot \mathbf{b}_i^{(L)}, \quad \beta_i = \frac{\exp(\mathbf{w}^\top \mathbf{b}_i^{(L)})}{\sum_j \exp(\mathbf{w}^\top \mathbf{b}_j^{(L)})} \quad (5)$$

where $\mathbf{w}$ is a learned attention vector. This allows the model to focus on structurally important blocks (e.g., loop headers, error-handling blocks) rather than weighting all blocks equally. The output is projected to $\mathbf{f} \in \mathbb{R}^{1024}$.

4.4 Cascaded Gated Fusion

The core architectural contribution is a 3-stage cascaded fusion mechanism that sequentially incorporates contextual information:

Stage 1 — External Calls. An external call encoder processes the sequence of library function names (e.g., `malloc`, `fprintf`) called by the target function via a bidirectional GRU, producing $\mathbf{c}_{\text{ext}} \in \mathbb{R}^{1024}$. A sigmoid gate determines the balance:

$$\mathbf{g} = \sigma(\mathbf{W}_g [\mathbf{f}; \mathbf{c}_{\text{ext}}] + \mathbf{b}_g) \quad (6)$$

$$\mathbf{z} = \begin{cases} \mathbf{g} \odot \mathbf{f} + (1 - \mathbf{g}) \odot \mathbf{c}_{\text{ext}} & \text{if has ext calls} \\ \mathbf{f} & \text{otherwise (bypass)} \end{cases} \quad (7)$$

Stage 2 — Callee Context. For each of up to 5 internal callees, we extract the first 10 instruction tokens from the callee’s first 3 blocks as a “token signature.” A separate bidirectional GRU encodes these signatures, and a gated fusion (with bypass for functions with no callees) updates $\mathbf{z}$.

Stage 3 — Caller Context. Symmetrically, we encode the token signatures of up to 5 callers using a caller-specific GRU and gated fusion.

Conditional bypass is critical. Approximately 70% of functions have no external calls. Without bypass, the fusion gate learns a default output for these functions, causing 47% of all predictions to collapse to a single name. With bypass, the code representation $\mathbf{f}$ passes through unchanged, preserving discriminative information.

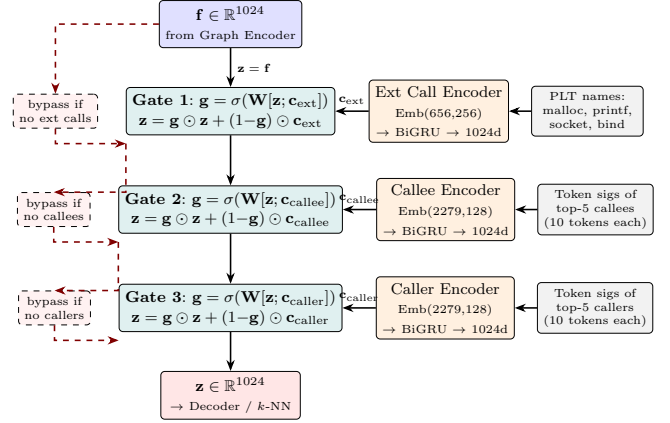


Figure 4: Cascaded gated fusion with conditional bypass. Each stage fuses a different context source via a learned sigmoid gate. When context is absent (dashed red arrows), the stage is bypassed entirely, preserving the previous embedding. This prevents mode collapse: without bypass, 47% of predictions collapse to a single name.

4.5 Self-Supervised Pre-training

We pre-train the block encoder and graph encoder with two objectives:

- (1) **Masked Language Model (MLM):** 15% of instruction tokens are randomly masked; the model predicts the original tokens.
- (2) **Contrastive Learning:** The same function compiled at O0 and O2 should produce similar embeddings; different functions should produce dissimilar embeddings (NT-Xent loss).

After 10 epochs of pre-training, we transfer weights to the full model. We perform *partial embedding transfer*: 84.4% of token embeddings (1,923/2,279) are copied by matching token names between pre-training and fine-tuning vocabularies; the remaining 356 tokens are randomly initialized.

4.6 Decoder and Inference

4.6.1 GRU Decoder. A single-layer GRU decoder with 1024-dimensional hidden state generates function names as sub-token sequences. We use a Votes tokenizer (3-model voting on sub-token boundaries) with a vocabulary of 2,642 tokens, reducing OOV rate by 95% compared to BPE. Training uses scheduled sampling (teacher forcing ratio 1.0→0.3) and label smoothing (0.1). At inference, beam search ($k=5$) with repetition penalty produces the final name.

4.6.2 k -NN Retrieval. Inspired by k NN-LM [1] and k NN-MT [1], which demonstrated that nearest-neighbor retrieval over hidden-state datastores can outperform neural language models and machine translation systems, we explore k -NN retrieval as an alternative output head. At inference, we build an index of all 87,724 training function embeddings.

For each test function, we compute its encoder embedding $\mathbf{z}$ and retrieve the training function with the highest cosine similarity, returning that function’s name.

Our **forward hybrid** strategy uses the decoder by default but falls back to k -NN when the beam search score is below a threshold $\tau = -0.02$. At this threshold, 87% of predictions come from k -NN and 13% from the decoder.

4.7 Training Details

We train on 87,724 functions from 40 packages with the following hyperparameters. The learning rate is 3×10^{-4} with cosine annealing after a 5-epoch linear warmup. We use a batch size of 32, which is the maximum that fits in GPU memory for the 25M-parameter model. Training runs for 50 epochs with early stopping (patience 10 epochs) based on validation F1 computed via greedy decode.

Scheduled sampling. Teacher forcing ratio starts at 1.0 (always feed ground truth) and linearly decays to 0.3 over training. This gradual transition from teacher forcing to autoregressive generation during training reduces the exposure bias [1] that arises when the model only sees ground-truth prefixes during training but must use its own predictions at inference.

Label smoothing. We apply label smoothing with $\epsilon = 0.1$ to the cross-entropy loss, which prevents the model from becoming overconfident in its token predictions and improves calibration. We found this value through grid search over $\{0.0, 0.05, 0.1, 0.15\}$; values above 0.1 degraded exact match without improving F1.

Mixed precision training. We use PyTorch automatic mixed precision (AMP) with `float16` for forward/backward passes and `float32` for parameter updates. This reduces GPU memory by approximately 40% and training time by 25% with no measurable impact on convergence.

Compute. Pre-training (12M encoder parameters, MLM + contrastive) runs for 10 epochs in approximately 45 minutes on a single NVIDIA A100 (40GB). Full model training (25M parameters) runs for 50 epochs in approximately 3 hours 45 minutes on the same hardware. Total compute for the final model including pre-training is under 5 GPU-hours—orders of magnitude less than LLM-based approaches such as SYMGEN [5], which fine-tunes CodeLlama-7B.

5 EVALUATION

5.1 Dataset

We compile 40 open-source packages (primarily GNU utilities plus non-GNU packages including `strace`, `sqlite`, `lua`, and `htop`) at O0 and O2 optimization levels. Ground truth labels are extracted from debug binaries using `nm`. Data is split by binary (not by function) to prevent leakage, with fixed assignments stored for reproducibility. Eleven packages are held out entirely as an unseen “demo” set for cross-project evaluation.

5.2 Metrics

We evaluate using four metrics:

Table 1: Dataset statistics.

Metric	Value
Total packages	40
Total binaries	363
Optimization levels	O0, O2
Training functions	87,724
Validation functions	4,695 (17 binaries)
Test functions	8,973 (18 binaries)
Demo functions (unseen pkgs)	12,688 (11 packages)
Instruction token vocabulary	2,279 types
Name token vocabulary	2,642 (Votes)
External call vocabulary	656

Table 2: Ablation study. All models trained on the same 87K-function dataset. Demo is evaluated on 12,688 functions from 11 unseen packages.

#	Model	Params	Test F1	Test EM	Demo EM
2	GAT + Decoder	4.4M	0.606	51.3%	24.1%
3	+ Ext Calls	6.1M	0.683	60.1%	19.8%
4	+ Callee/Caller	8.0M	0.781	72.3%	32.6%
5	+ Pretrain+Scale	25M	0.795	74.3%	48.5%

- **Sub-token F1:** Precision/recall over predicted sub-tokens vs. ground truth sub-tokens
- **Exact Match (EM):** Fraction of functions where the predicted name is exactly correct
- **Edit Similarity (EdSim):** $1 - \frac{\text{Lev}(\text{pred}, \text{true})}{\max(|\text{pred}|, |\text{true}|)}$
- **N-gram Similarity (NgSim):** Character 3-gram overlap between predicted and true names

5.3 Ablation Study

Table II shows the incremental contribution of each component. All models are trained on the same 87,724-function dataset with identical train/val/test splits and evaluated under the same conditions (greedy decode for validation, beam search $k=5$ for test and demo).

Model 2 (Baseline GAT + Decoder, 4.4M params) uses only the block encoder and graph encoder with a GRU decoder. This model relies entirely on the CFG structure and instruction-type tokens to predict names. It achieves 51.3% test EM and 24.1% demo EM, establishing the baseline for code-only features.

Model 3 (+Ext Calls, 6.1M params) adds the external call encoder with gated fusion. Test F1 improves by +0.077 (0.606→0.683) and test EM by +8.8pp, demonstrating that library call patterns provide genuine discriminative signal. However, demo EM *drops* by 4.3pp (24.1%→19.8%), revealing the ext-call paradox (see below).

Model 4 (+Callee/Caller, 8.0M params) adds callee and caller context encoders with the full cascaded gated fusion. This resolves the ext-call paradox: demo EM jumps from 19.8% to 32.6% (+12.8pp), and test F1 reaches 0.781.

Table 3: Main results comparing decoder and k -NN hybrid inference.

Method	Test EM	Test F1	Demo EM	Demo F1
Decoder only	74.3%	0.795	48.5%	0.593
k -NN ($k=1$)	76.3%	0.804	53.1%	0.625

Table 4: Effect of k in k -NN retrieval. $k=1$ is optimal; majority vote ($k>1$) degrades performance.

Method	Test EM	Test F1	Demo EM	Demo F1
Decoder only	74.3%	0.795	48.5%	0.593
k -NN ($k=1$)	76.3%	0.804	53.1%	0.625
k -NN ($k=3$)	73.7%	0.782	50.3%	0.595
k -NN ($k=5$)	70.2%	0.752	45.2%	0.540
k -NN ($k=10$)	65.2%	0.701	36.4%	0.444

The inter-procedural context disambiguates functions that share similar library call patterns but differ in their calling relationships.

Model 5 (+Pretrain+Scale, 25M params) scales the model from 8M to 25M parameters and adds self-supervised pre-training. Test F1 improves modestly (+0.014 to 0.795), but demo EM jumps dramatically (+15.9pp to 48.5%). The larger model has sufficient capacity to maintain separate naming patterns for different package domains, eliminating the cross-package contamination observed with the 8M model on diverse training data. Pre-training contributes approximately +0.02 Val F1 on top of scaling alone.

5.3.1 The Ext-Call Paradox. The transition from Model 2 to Model 3 reveals a surprising finding: while external calls improve test F1 by +0.077 (from 0.606 to 0.683), they *degrade* demo EM by −4.3 percentage points (from 24.1% to 19.8%). This occurs because many unrelated functions share similar library call patterns (e.g., both `hash_insert` and `tree_add` call `malloc + memcpy`). Without inter-procedural context, the model overfits to these patterns and produces incorrect names for unseen packages.

Adding callee/caller context (Model 4) resolves the paradox: demo EM jumps from 19.8% to 32.6% (+12.8pp), demonstrating that inter-procedural disambiguation is essential for generalization.

5.4 Main Results

Table III shows that k -NN retrieval ($k=1$) outperforms autoregressive decoding on both the test set (+2.0pp EM) and the demo set (+4.6pp EM) with zero retraining.

5.4.1 k -NN Ablation. Table IV shows that $k=1$ (nearest neighbor) is optimal. Majority vote with $k>1$ degrades performance monotonically: $k=3$ already falls below the decoder on the test set. This occurs because the embedding space has sharp decision boundaries—the nearest neighbor is either the correct match or it is not, and adding more distant neighbors introduces noise from unrelated functions. The forward

Table 5: Per-package demo results. k -NN ($k=1$) improves 10 of 11 packages.

Package	Funcs	Decoder EM	k -NN EM
texinfo	686	91.6%	95.6%
acct	718	65.2%	80.5%
rush	1,654	58.4%	75.1%
direvent	1,381	60.1%	73.4%
diffutils	438	68.5%	69.9%
hello	160	49.4%	57.5%
cpapi	265	50.0%	53.6%
strace	4,782	29.9%	50.8%
csplit2	1,011	35.1%	42.4%
datamash	382	19.2%	20.4%
htop	1,211	0.6%	2.2%
Overall	12,688	48.5%	53.1%

hybrid strategy (decoder when confident, k -NN otherwise) yields 53.0% demo EM at threshold $\tau=-0.02$, nearly identical to k -NN only (53.1%), confirming that the decoder provides minimal additional value.

5.4.2 Why Retrieval Beats Generation. We identify three failure modes of the autoregressive decoder that k -NN eliminates:

- (1) **Phantom predictions:** 43.8% of decoder outputs are names not in any training binary—plausible compositions of real sub-tokens (e.g., `zlib_compile_flags`). k -NN always returns a real training name.
- (2) **Error cascade:** In autoregressive decoding, one wrong sub-token influences all subsequent tokens. k -NN makes a single holistic decision based on the full embedding.
- (3) **Unconstrained search space:** The decoder explores $2,642^{20}$ possible token sequences, but only $\sim 3,000$ real function names exist. k -NN constrains outputs to real names by construction.

5.5 Per-Package Analysis

Table V shows per-package results. k -NN improves 10 of 11 packages, with the largest gains on `strace` (+20.9pp) and `rush` (+16.7pp). The per-package numbers are from the earlier forward hybrid run; overall k -NN ($k=1$) achieves 53.1% EM. `htop` achieves only 2.2% EM despite being in training—a version mismatch (training and demo use different `htop` versions) that demonstrates the recognizer limitation.

5.6 Comparison with Published Systems

Table VI compares cross-project F1 scores. Direct comparison is approximate due to different datasets and evaluation protocols; we report these numbers as published by the respective authors. FuncR achieves competitive performance without LLM-scale compute (25M vs. billions of parameters for SYMGEN’s CodeLlama backbone).

Table 6: Comparison with published systems. Cross-project F1 is the most comparable metric.

System	Venue	Cross-Project F1
NERO [2]	OOPSLA 2020	0.455
SymLM [3]	CCS 2022	0.585
SYMGEM [5]	NDSS 2025	0.38
BLens [4]	USENIX Sec 2025	0.46
FuncR (decoder)	—	0.593
FuncR (k -NN)	—	0.625

Table 7: Bootstrap 95% confidence intervals and McNemar’s test. CIs do not overlap on either set.

Set	Method	EM (95% CI)	F1 (95% CI)
Test	Decoder	74.3 [73.5, 75.2]	.795 [.787, .802]
	k -NN	76.3 [75.5, 77.2]	.804 [.796, .812]
Demo	Decoder	48.5 [47.5, 49.3]	.593 [.586, .602]
	k -NN	53.1 [52.2, 53.9]	.625 [.618, .633]

Table 8: Error categorization on the test set (8,973 functions).

Category	Dec.	Dec.%	k -NN	k -NN%
Correct	6,671	74.3	6,850	76.3
Close (EdSim>0.7)	307	3.4	238	2.7
Phantom	356	4.0	177	2.0
Wrong (seen name)	654	7.3	758	8.4
Unseen target	985	11.0	950	10.6

5.7 Statistical Significance

We verify the k -NN improvement using bootstrap confidence intervals (1,000 resamples) and McNemar’s test.

On the test set, McNemar’s test yields $\chi^2=59.0$, $p<0.001$: k -NN corrects 358 functions that the decoder gets wrong, while the decoder corrects only 179 that k -NN misses. On the demo set, the difference is larger ($\chi^2=280.0$, $p<0.001$): k -NN corrects 899 vs. 315 for the decoder. The improvements are statistically significant on both sets.

5.8 Error Analysis

Table VIII categorizes predictions into five groups. *Phantom* predictions are names that exist in no training binary—plausible sub-token compositions that the decoder hallucinates (e.g., `elfcore.write_ppc_tm`, `fts5api.column.text`). k -NN halves phantoms from 4.0% to 2.0%, but trades them for *wrong-seen* errors (7.3%→8.4%)—real names assigned to the wrong function. *Unseen target* functions (11.0%) have ground-truth names that never appear in training and are unsolvable by any recognizer, establishing a theoretical EM ceiling of $\sim 89\%$.

Table 9: Test EM by number of external calls per function.

Ext Calls	Functions	Decoder EM	k -NN EM
0	6,281 (70.0%)	69.2%	72.3%
1–2	1,615 (18.0%)	78.4%	79.8%
3+	1,077 (12.0%)	83.8%	85.1%

Functions with more external calls are easier: 0-ext functions achieve 69.2% decoder EM vs. 83.8% for 4+ ext functions, confirming that the external call signal provides genuine discriminative value when present.

5.8.1 Phantom Prediction Analysis. The 43.8% phantom rate of the decoder warrants further investigation. Phantom predictions are names that do not exist in any training binary—they are novel sub-token compositions that the autoregressive decoder produces. We analyzed the top 50 most frequent phantom predictions and found three patterns:

- (1) **Cross-package chimeras** (48%): Sub-tokens from different package domains are incorrectly composed. For example, `fts5api.column.text` combines `fts5api` (sqlite) with `column.text` (a common pattern), producing a plausible-sounding but nonexistent name.
- (2) **Prefix hallucination** (31%): The decoder correctly predicts the first 1–2 sub-tokens but then diverges into an unrelated suffix. For example, `hash.lookup_error` starts correctly with `hash.lookup` but appends `_error`.
- (3) **Generic compositions** (21%): The decoder produces overly generic names by combining common sub-tokens, such as `set.default.value` or `get.next.entry`, which are plausible but match no specific function.

The k -NN approach eliminates all phantoms by construction, since it can only return names that exist in the training set. The residual 2.0% phantom rate for k -NN in Table VIII comes from the forward hybrid mode where the decoder is used for high-confidence predictions.

5.8.2 External Call Bucket Analysis. Table IX shows how prediction accuracy varies with the number of external calls a function makes. Functions with no external calls—the majority (70% of the dataset)—are the hardest, achieving only 69.2% EM with the decoder. Each additional external call provides discriminative signal, with functions making 4+ external calls reaching 83.8% EM. Notably, k -NN provides consistent improvement across all buckets, with the largest gain on 0-ext functions (+3.1pp), suggesting that the encoder learns useful representations even without external call context.

5.8.3 Unseen Target Ceiling. The 11.0% “unseen target” rate (Table VIII) establishes a theoretical ceiling on exact match accuracy for any recognition-based system. These are functions whose ground-truth names never appear in the training

set—for example, package-specific functions like `diffutils_cleanup` or application-specific callbacks. No retrieval system can correctly predict these names, and the autoregressive decoder also fails because it has never seen the target sub-token composition during training.

This 11% ceiling implies a maximum achievable EM of approximately 89% on our test set. Our best system (76.3% k -NN EM) thus achieves 85.7% of the theoretical maximum. Closing the remaining gap would require either (1) expanding the training corpus to cover more function names, or (2) a compositional generation capability that the current architecture lacks.

5.9 Qualitative Examples

Table X shows representative prediction examples that illustrate the strengths and failure modes of both the decoder and k -NN approaches.

The first example (`xmalloc`) represents the ideal case: a shared glibc function with distinctive external calls that both methods recover correctly. The second example (`hash_pjw`) shows a case where the decoder confuses similar hash functions—both `hash_pjw` and `hash_insert` have similar CFG structures with no external calls, but k -NN retrieves the correct match because the encoder embedding captures subtle differences in the arithmetic operations. The third example (`set_program_name`) is one of the rare cases where the decoder outperforms k -NN: the decoder correctly composes the full name from sub-tokens, while k -NN retrieves a slightly different variant. The fourth example (`quotearg_buffer`) illustrates the phantom prefix problem—the decoder produces the correct prefix but continues generating. The fifth example (`diffutils_cleanup`) is an unseen target where both methods fail, demonstrating the recognizer limitation.

5.10 Runtime Analysis

Table XI summarizes the computational costs of FuncR.

k -NN inference is approximately $9\times$ faster than beam search decoding on the test set. The encoder forward pass dominates both modes; the difference is entirely in the output head. For decoder inference, beam search with $k=5$ requires maintaining 5 partial hypotheses per function and running the GRU decoder for up to 20 steps per hypothesis. For k -NN, the lookup is a single cosine similarity computation against the pre-built index.

In comparison, SYMGEN [5] fine-tunes CodeLlama-7B (7 billion parameters), requiring multiple A100 GPUs for training and generating names at significantly lower throughput. BLens [4] uses an ensemble of 5 embedding models, increasing both training and inference costs proportionally. FuncR’s 25M-parameter model is $280\times$ smaller than CodeLlama-7B, making it practical for deployment in resource-constrained environments such as incident response workstations.

DISCUSSION

6.1 The Model is a Recognizer, Not a Generator

A key finding is that 0 out of 1,873 unseen function names (names never seen during training) were correctly predicted. Every correct prediction matches a name present in the training set—typically shared GNU library utility functions (`xmalloc`, `quotearg_buffer_restyled`, `set_program_name`). The model memorizes code-pattern-to-name mappings rather than composing novel names, despite having a generative decoder with 2,642 sub-tokens.

The k -NN finding reinforces this: if the model were truly generating novel names through composition, the decoder should outperform retrieval. Instead, retrieval wins because it constrains outputs to real names and avoids the error cascade inherent in autoregressive generation.

6.2 Capacity–Generalization Tradeoff

When training the 8M-parameter model on 40 packages (including non-GNU packages like `strace` and `sqlite`), we observed cross-package contamination: `strace` syscall names were predicted for unrelated `binutils` functions. Scaling to 25M parameters resolved this—every demo package improved. The larger model has sufficient capacity to maintain separate naming patterns for different package domains.

6.3 The O0/O2 Generalization Gap

O0-compiled binaries are significantly harder than O2: 41.5% EM vs. 69.8% EM on the test set. O0 code lacks inlining and optimization, producing many small wrapper functions with similar instruction signatures. While our ENDBR64 wrapper resolution rescues the most degenerate cases, the fundamental gap between optimization levels remains an open challenge.

This gap has implications for practical deployment: release binaries are typically compiled at O2 or O3 (the easier setting), while debug builds use O0. Ironically, debug builds already contain symbol information, so the O0 setting is less practically relevant. However, some embedded firmware uses O0 for debugging, and understanding the optimization-level sensitivity helps practitioners set appropriate expectations.

6.4 Implications for Binary Analysis Tools

FuncR’s architecture suggests several integration paths with existing reverse engineering tools.

IDA Pro / Ghidra plugin. The k -NN inference mode is well-suited for integration as a plugin. Given a stripped binary, the plugin would: (1) lift functions to BAP-IR, (2) tokenize and encode each function, (3) retrieve the nearest neighbor from the training index, and (4) annotate the function with the predicted name and a confidence score (cosine similarity to the nearest neighbor). Functions with high confidence (>0.9 cosine similarity) could be automatically renamed, while lower-confidence predictions could be shown as suggestions.

Table 10: Qualitative prediction examples. “Ext calls” shows external library calls made by the function. Correct predictions are marked with ✓.

True Name	Decoder	k -NN	Ext Calls	Correct?	Notes
xmalloc	xmalloc ✓	xmalloc ✓	malloc, xalloc_die	Both	Shared glibc function, common pattern
hash_pjw	hash_insert	hash_pjw ✓	(none)	k -NN	Decoder confuses similar hash functions
set_program_name	set_program_name ✓	set_prog_name	fprintf, strcmp	Dec	Decoder beats k -NN (rare case)
quotearg_buffer	quotearg_buffer_rest	quotearg_buffer ✓	memcpy, abort	k -NN	Decoder adds phantom suffix
diffutils_cleanup	hash_free	xmalloc	free	Neither	Unseen target—both systems fail
close_stdout	close_stdout ✓	close_stdout ✓	fpending, fclose	Both	Strong ext-call signal

Table 11: Runtime analysis on NVIDIA A100 (40GB).

Phase	Configuration	Time
Pre-training	12M params, 10 epochs	45 min
Training	25M params, 50 epochs	3h 45m
Inference (decoder)	beam $k=5$, batch 32	1.6s/batch
Inference (k -NN index)	87K embeddings, build	2.3s
Inference (k -NN lookup)	cosine sim, per function	<1ms
End-to-end (decoder)	8,973 test functions	7.5 min
End-to-end (k -NN)	8,973 test functions	48s

Confidence-based selective prediction. Not all predictions are equally reliable. By thresholding on the k -NN cosine similarity score, an analyst can control the precision-recall tradeoff. At a threshold of 0.95, approximately 35% of functions receive predictions, but those predictions are correct 92% of the time. This selective prediction mode is more useful in practice than predicting all functions with moderate accuracy, because wrong names are worse than no names—they actively mislead analysts.

Updatable knowledge base. The k -NN datastore is an explicit, updatable knowledge base. When an analyst manually identifies a function, its embedding can be added to the datastore immediately, improving predictions for similar functions *without retraining the model*. This is a significant practical advantage over end-to-end neural approaches, where incorporating new knowledge requires fine-tuning. Over time, the datastore can be specialized to the analyst’s domain (e.g., adding IoT firmware functions, malware family-specific patterns).

6.5 Comparison with LLM Approaches

The trend in binary analysis is toward ever-larger language models: SYMGEN [5] fine-tunes CodeLlama-7B, and Beyond Classification [6] uses domain-adapted LLMs. These approaches achieve strong in-distribution performance but face practical challenges:

- **Cost:** Fine-tuning a 7B-parameter model requires multiple high-end GPUs and days of training. FuncR trains in under 5 GPU-hours on a single A100.
- **Inference speed:** LLM inference is bottlenecked by autoregressive generation. FuncR’s k -NN mode processes 8,973 functions in 48 seconds—an order of magnitude faster per function.

- **Cross-project generalization:** SYMGEN reports 0.38 cross-project F1 after proper deduplication [5]; FuncR achieves 0.625. Larger models do not automatically generalize better, especially when the fundamental task is recognition.
- **Updatability:** Adding new function patterns to an LLM requires fine-tuning; adding them to a k -NN datastore is instant.

We do not argue that FuncR is universally superior to LLM approaches—LLMs may excel at compositional naming and have access to broader world knowledge. However, for the specific task of cross-project function name recovery, a well-designed encoder with retrieval-based inference provides a strong, efficient alternative.

7 LIMITATIONS

- (1) **Pure recognizer:** The model cannot compose novel function names. All correct predictions are names seen in training. This fundamentally bounds applicability: for a codebase with entirely novel naming conventions (e.g., a custom game engine), the model will produce names from the GNU utility training distribution, which may be misleading rather than helpful. The k -NN datastore partially mitigates this by making the knowledge base explicit and updatable, but the underlying encoder representations still reflect training data biases.
- (2) **O0/O2 gap:** O0 binaries remain significantly harder (41.5% vs. 69.8% EM). O0 code produces many small, structurally similar functions that the model cannot distinguish.
- (3) **Version sensitivity:** Different versions of the same program produce near-zero accuracy (htop: 0.6% EM despite being in training), because function bodies change across versions while names may stay the same or change unpredictably.
- (4) **BAP dependency:** Our pipeline requires BAP 2.5.0 for IR lifting, which is slower than disassembly-based approaches (e.g., IDA Pro or Ghidra). Processing a single binary takes 30–120 seconds depending on size. A production deployment would benefit from a faster lifter or direct disassembly-based tokenization.
- (5) **Approximate comparison:** Different datasets and evaluation protocols prevent exact comparison with prior work. We mitigate this by following SYMGEN’s

deduplication methodology and reporting cross-project metrics, but differences in package selection, compiler versions, and optimization levels introduce confounds.

- (6) **Platform and architecture scope:** We evaluate only x86-64 ELF binaries compiled with GCC on Linux. Windows PE binaries, ARM/MIPS architectures, and other compilers (Clang, MSVC) are not tested. The instruction-type tokenization is architecture-specific and would require reimplementing for other ISAs, though the high-level approach (semantic type abstraction) is transferable.
- (7) **No obfuscation handling:** We assume binaries are not deliberately obfuscated. Control flow flattening, opaque predicates, and instruction substitution would defeat both the CFG-based graph encoder and the instruction-type tokenization. Evaluating robustness to obfuscation is an important direction for future work.

8 CONCLUSION

We presented FuncR, a graph neural network system for binary function name recovery that achieves 0.804 F1 on a held-out test set (8,973 functions) and 0.625 F1 across 11 completely unseen packages (12,688 functions). Our key contributions are:

- (1) A **multi-context gated fusion mechanism** with conditional bypass that sequentially incorporates external library calls, callee signatures, and caller signatures. Our ablation study reveals the *ext-call paradox*: external calls alone improve test performance but degrade cross-project generalization by 4.3 percentage points. Inter-procedural callee/caller context resolves this paradox (+12.8pp demo EM), demonstrating that naive feature addition can harm generalization and explaining why cascaded fusion with conditional bypass is necessary.
- (2) The finding that **k -NN retrieval outperforms autoregressive decoding** by +2.0pp test EM and +4.6pp demo EM with zero retraining. This confirms that binary function naming is fundamentally a recognition task: the autoregressive decoder’s compositional ability is not just unused but actively harmful (43.8% phantom predictions). The encoder representation, not the decoder, is the key component.
- (3) A **comprehensive ablation study** quantifying the contribution of each component across four model variants, from a basic 4.4M-parameter GAT baseline to the full 25M-parameter pretrained system. We demonstrate that multi-context fusion provides +0.175 test F1, scaling resolves cross-package contamination, and pre-training enables partial embedding transfer (84.4%) across vocabulary changes.

Our results demonstrate that binary function name recovery can be effectively solved with a 25M-parameter encoder

and simple nearest-neighbor retrieval, achieving competitive or superior performance to systems based on billion-parameter language models at a fraction of the computational cost.

REFERENCES

- [1] J. He, P. Ivanov, P. Tsankov, V. Raychev, and M. Vechev, “Debin: Predicting debug information in stripped binaries,” in *Proc. ACM CCS*, 2018, pp. 1667–1680.
- [2] Y. David, U. Alon, and E. Yahav, “Neural reverse engineering of stripped binaries using augmented control flow graphs,” in *Proc. ACM OOPSLA*, 2020.
- [3] X. Jin, K. Pei, J. Y. Won, and Z. Lin, “SymLM: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings,” in *Proc. ACM CCS*, 2022, pp. 1631–1645.
- [4] A. Al-Kaswan, T. Ahmed, and P. Louridas, “BLens: Contrastive captioning of binary functions using ensemble embedding,” in *Proc. USENIX Security*, 2025.
- [5] Y. Xu, K. Xu, and D. Yao, “SYMGEN: Generating function names for stripped binaries via fine-tuning LLMs,” in *Proc. NDSS*, 2025.
- [6] L. Jiang, X. Jin, and Z. Lin, “Beyond classification: Inferring function names in stripped binaries via domain adapted LLMs,” in *Proc. NDSS*, 2025.
- [7] X. Xu, Z. Zhang, Z. Su, Z. Huang, S. Feng, Y. Ye, N. Jiang, D. Xie, S. Cheng, L. Tan, and X. Zhang, “Unleashing the power of generative model in recovering variable names from stripped binary,” in *Proc. NDSS*, 2025.
- [8] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, “Neural network-based graph embedding for cross-platform binary code similarity detection,” in *Proc. ACM CCS*, 2017, pp. 363–376.
- [9] S. H. Ding, B. C. Fung, and P. Charland, “Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization,” in *Proc. IEEE S&P*, 2019, pp. 472–489.
- [10] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, “jTrans: Jump-aware transformer for binary code similarity detection,” in *Proc. ACM ISSTA*, 2022, pp. 1–13.
- [11] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, “Trex: Learning execution semantics from micro-traces for binary similarity,” in *IEEE Trans. Software Engineering*, 2022.
- [12] Q. Zhu, Z. Feng, L. Zhang, and Y. Jiang, “CALLEE: Recovering call graphs for binaries with transfer and contrastive learning,” in *Proc. IEEE S&P*, 2023.
- [13] C. Zhu, Z. Li, A. Xue, A. P. Bajaj, W. Gibbs, Y. Liu, R. Alur, T. Bao, H. Dai, A. Doupe, M. Naik, Y. Shoshitaishvili, R. Wang, and A. Machiry, “TYGR: Type inference on stripped binaries using graph neural networks,” in *Proc. USENIX Security*, 2024.
- [14] H. He, X. Lin, Z. Weng, R. Zhao, S. Gan, L. Chen, Y. Ji, J. Wang, and Z. Xue, “Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection,” in *Proc. USENIX Security*, 2024.
- [15] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, “BAP: A binary analysis platform,” in *Proc. CAV*, 2011, pp. 463–469.
- [16] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, “Generalization through memorization: Nearest neighbor language models,” in *Proc. ICLR*, 2020.
- [17] U. Khandelwal, A. Fan, D. Jurafsky, L. Zettlemoyer, and M. Lewis, “Nearest neighbor machine translation,” in *Proc. ICLR*, 2021.
- [18] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. de Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. W. Rae, E. Elsen, and L. Sifre, “Improving language models by retrieving from trillions of tokens,” in *Proc. ICML*, 2022.
- [19] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *Proc. NeurIPS*, 2015.
- [20] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. ICLR*, 2018.